Consensus-Based controller for warehouse robots task distribution

Andy Tsai
1233653410

Alan Cheng
1226164539

Pin-Chi Chen
1234881468

***Abstract*—This project investigates distributed task allocation in multi-robot systems using a consensus-based supervisory control framework. Rather than coordinating continuous motion, robots negotiate discrete task assignments using an iterative Max-Consensus algorithm. This behavior is relevant to large-scale warehouse automation, search-and-rescue coordination, and multi-robot inspection, where independent agents must efficiently distribute tasks while maintaining robustness to communication constraints.**

The system is modeled as a network of agents that exchange task-bid information, where each robot computes a bid based on its distance from the current task and updates its belief according to local communication. The model captures the discrete decision dynamics of multi-robot auctions, and the analysis focuses on two core properties: finite-time convergence of the Max-Consensus process and conflict-free task allocation enforced by sequential capacity constraints. These properties are examined through graph-theoretic tools and known convergence bounds related to the network diameter.

The simulation results validate the theoretical analysis. The robots reliably converge on a winner for each sequential auction, ensuring all tasks are assigned to distinct agents without conflict. After allocation, navigation goals are dispatched through the Nav2 stack, demonstrating a complete perception–decision–action pipeline. The results highlight that consensus-based task allocation offers a scalable and mathematically provable alternative to centralized planning methods.

I. Introduction

Multi-robot systems have become increasingly important in modern industrial and service applications, including warehouse automation, coordinated inspection, and search-and-rescue operations. A fundamental challenge in these systems is the assignment of spatially distributed tasks to autonomous robots in a manner that is scalable, robust to communication limitations, and computationally efficient. Traditional centralized task allocation architectures can compute optimal solutions, but they suffer from high communication overhead, reliance on a single point of failure, and limited adaptability to dynamic environments. These limitations motivate the use of distributed decision-making strategies that rely only on local interactions among robots.

Among distributed approaches, consensus algorithms represent a powerful and mathematically grounded framework for achieving coordinated multi-agent behavior. Consensus-based methods have been extensively analyzed in the fields of cooperative control and graph theory [1], [2]. While many prior works focus on consensus for continuous-state variables such as positions, velocities, or formation parameters, consensus mechanisms can also be used to coordinate discrete decisions. In particular, task allocation can be formulated as a consensus process in which robots collectively agree on the "winning" robot for each task based on individual task-related costs.

This work investigates a distributed task allocation algorithm based on **an iterative** Max-Consensus protocol. Each robot computes a bid for **the current active** task based on its Euclidean distance and exchanges this information with neighbors over a fixed communication graph. Through repeated application of the max-update rule, the highest bid propagates through the network, and all robots eventually agree on the winning robot. **To ensure a balanced distribution of labor, the algorithm assigns tasks sequentially, removing the winning robot from the pool of available agents for subsequent tasks.** This approach naturally enables conflict-free assignments, requires no centralized coordination, and provides finite-time convergence guarantees rooted in algebraic graph theory.

The proposed method is motivated by practical warehouse scenarios in which robots operate in a bounded indoor environment with reliable short-range communication. By separating the high-level decision layer from the low-level motion layer, the method integrates seamlessly with existing navigation architectures such as Nav2. Once the distributed consensus process converges, each robot independently executes the assigned task by issuing a navigation goal to its motion planner.

The main contributions of this work are summarized as follows. First, we develop a mathematically explicit model of distributed task allocation based on bid dynamics and neighborhood-based information exchange. Second, we provide rigorous proofs of two key properties of the system: (i) finite-time convergence of the Max-Consensus process, and (ii) **guaranteed uniqueness of task assignment under sequential capacity constraints**. These results rely on established theoretical tools in graph theory and monotone dynamical systems [3]. Finally, we validate the collective behavior through simulation, demonstrating that robots consistently achieve correct agreements and produce conflict-free task distributions without centralized control.

II. MATHEMATICAL MODEL

Team contributions: Andy Tsai, Pin-Chi Chen

A. Multi-robot behavior and application scenario

We investigate a consensus-based task allocation behavior in a team of mobile robots operating in an indoor warehouse environment. A set of spatially distributed tasks must be managed by a team of autonomous robots. Each task corresponds to, for example, picking an item from a shelf, inspecting a storage location, or delivering a payload to a docking station.

The objective is to assign each task to exactly one robot in a distributed manner while approximately minimizing the total travel distance of the team. Instead of coordinating continuous motion at the velocity level, we model and analyze the discrete decision dynamics that governs which robot is assigned to which task. The robots exchange local information over a communication network and agree on winning bids for all tasks using a Max-Consensus

protocol. The winning robots then use a standard navigation stack (Nav2) to execute the corresponding motion plans. This separation isolates the mathematically tractable distributed decision layer from the low-level motion layer.

B. Assumptions and constraints

We summarize the assumptions and constraints used in the model:

- **Robots and the environment:** The workspace is a rectangular bounded warehouse $\Omega \subset \mathbb{R}^2$ with static obstacles (shelves, walls, etc.). Robots move on a planar surface and are modeled at the decision layer as point agents with positions $p_i \in \mathbb{R}^2$, $i \in \{1, ..., N\}$. Low-level dynamics and obstacle avoidance are handled by the Nav2 stack and are abstracted away.
- **Sensing and localization:** Each robot has access to its own pose estimate $p_i(t)$ from onboard localization (e.g., SLAM or AMCL) and a global map. The position of each task t_k is known and denoted by $q_k \in \mathbb{R}^2$, $k \in \{1, ..., M\}$.
- **Communication graph:** Robots exchange information over a fixed, undirected, and connected communication graph $G = (V, E)$, where $V = \{1, ..., N\}$ is the set of robots and $(i, j) \in E$ if robots i and j can communicate. The set of neighboring robots i is denoted by $\mathcal{N}_i = \{j \in V | (i, j) \in E\}$.
- **Communication and update model:** Time is discretized into synchronous decision steps $l = 0, 1, 2, \ldots$. At each step, robots transmit their local task-bid information to neighbors and update their internal state according to a Max-Consensus rule. Communication is assumed to be reliable and delay-free on the decision timescale.
- **Task model:** Each task t_k is static and must be assigned to exactly one robot. In this project, we consider single-robot tasks (no coalitions).

C. Variables and parameters

We now define the variables and parameters of the model:

- N: number of robots, indexed by $i \in \{1, ..., N\}$.
- M: number of tasks, indexed by $k \in \{1, ..., M\}$.
- $p_i(l) \in \mathbb{R}^2$: planar position of robot i at decision step l.
- $q_k \in \mathbb{R}^2$: planar position of task k.
- $c_{ik}(l) \in \mathbb{R}_{\geq 0}$: cost of assigning robot i to task k at step l, modeled as the Euclidean distance:

$$c_{ik}(l) = ||p_i(l) - q_k||_2 \tag{1}$$

- $b_{ik}(l)$: local bid of robot i for task k at step l, defined as a decreasing function of the cost:

$$b_{ik}(l) = \frac{1}{c_{ik}(l) + \epsilon} \tag{2}$$

  where $\epsilon > 0$ is a small regularization constant.
- $y_{ik}(l)$: entry of the winning bid vector of robot i for task k at step l, representing the highest bid for task k that robot i currently knows.
- $w_{ik}(l)$: index of the robot that robot i currently believes is the winner of task k at step l.
- $a_{ik} \in \{0, 1\}$: final assignment decision, where $a_{ik} = 1$ if robot i is assigned to task k, and 0 otherwise.

D. Consensus-based task allocation model

The collective decision-making behavior is modeled as a Max-Consensus process on the task-bid information. At each decision step l, robot i performs the following operations:

1) Local bid computation: Using its current position $p_i(l)$ and the known task locations q_k, robot i computes the local bid $b_{ik}(l)$. If $b_{ik}(l)$ is larger than its currently stored winning bid $y_{ik}(l)$, robot i updates its local winning information:

$$\text{if } b_{ik}(l) > y_{ik}(l) \text{ then } y_{ik}(l) \leftarrow b_{ik}(l),\ w_{ik}(l) \leftarrow i. \tag{3}$$

2) Neighbor communication: Robot i broadcasts the pair $(y_{ik}(l), w_{ik}(l))$ for all k to its neighbors $j \in \mathcal{N}_i$ and receives the corresponding information from them.

3) Max-Consensus update: For each task k, robot i updates its winning bid and winner index by selecting the maximum bid among itself and its neighbors:

$$(y_{ik}(l+1), w_{ik}(l+1)) = \arg \max_{j \in \mathcal{N}_i \cup \{i\}} y_{jk}(l) \tag{4}$$

with a deterministic tie-breaking rule (e.g., the smallest robot index) when multiple robots share the same maximum bid.

E. Iterative Assignment Strategy

To ensure a balanced distribution of labor and enforce a strict capacity constraint (i.e., one task per robot), the task allocation is performed sequentially rather than in parallel. We define an availability set $\mathcal{A}_k \subseteq V$, representing the set of robots available to bid on task k.

1) **Initialization:** For the first task ($k = 1$), all robots are available: $\mathcal{A}_1 = V$.
2) **Consensus Execution:** The Max-Consensus protocol defined in Eq. (5) is executed for task k using only the robots in $\mathcal{A}_k$. Robots $j \notin \mathcal{A}_k$ do not participate (or effectively transmit a bid of -1).
3) **Winner Removal:** Upon convergence, the winner i^* of task k is identified. This robot is then removed from the availability set for all subsequent tasks:

$$\mathcal{A}_{k+1} = \mathcal{A}_k \setminus \{i^*\} \tag{5}$$

4) **Iteration:** The process repeats for task $k + 1$ until all M tasks are assigned. This guarantees that no robot is assigned more than one task, provided $M \leq N$.

F. Realism of the model

The proposed model captures the key features of distributed task allocation in many realistic multi-robot applications. The communication graph and Max-Consensus dynamics are consistent with widely studied agreement protocols on graphs [1], [2]. Modeling the bid as a decreasing function of travel distance is a natural choice for logistics and inspection tasks, where minimizing the total distance or time is a primary objective. The abstraction that delegates low-level motion and collision avoidance to an existing navigation stack reflects how practical systems are built, while keeping the mathematical analysis focused on the collective decision-making layer.

III. THEORETICAL ANALYSIS

Team contributions: Andy Tsai, Pin-Chi Chen

A. Overview

This section presents a rigorous theoretical analysis of the Max-Consensus task allocation model defined in Section II. We prove two key properties of the system:

1) Finite-time convergence of the winning-bid dynamics for each auction.
2) Conflict-free task allocation under the iterative assignment strategy.

Both results rely on standard tools from algebraic graph theory and monotone dynamical systems [1]-[3].

B. Preliminaries

Recall the winning-bid update law for robot i and task k:

$$y_{ik}(l+1) = \max_{j \in \mathcal{N}_i \cup \{i\}} y_{jk}(l) \tag{6}$$

Define the stacked winning-bid vector $y_k(l) = [y_{1k}(l), \ldots, y_{Nk}(l)]^T$, which represents the network's collective belief about the highest bid on task k. The update is equivalent to the iteration $y_k(l+1) = M_G(y_k(l))$, where $M_G(\cdot)$ is a monotone, max-preserving operator induced by the adjacency structure of G.

We assume throughout that the communication graph G is undirected, connected, and static, and that robots perform synchronous updates. These assumptions satisfy the conditions of standard convergence theorems for max-consensus [1], [2].

C. Property 1: Finite-time convergence

Let the initial global maximum for a specific task k among the available robots be $y_k^* = \max_{i \in \mathcal{A}_k} y_{ik}(0)$.

[Theorem 1] For each task k, the max-consensus iteration converges in at most D steps, where D is the diameter of the communication graph G. Specifically:

$$y_{ik}(l) = y_k^* \quad \forall i \in V,\ \forall l \geq D. \tag{7}$$

Proof: Since G is connected, the diameter D is the longest shortest-path length between any two robots. Consider the robot i^* that holds the global maximum y_k^* initially. At iteration $l = 1$, all neighbors of i^* adopt the value y_k^* because of the max-update rule. By induction, the set of robots holding y_k^* grows by at least one hop per iteration. Thus, after D iterations, every robot in the connected component is within distance D of i^*, and therefore all robots hold y_k^*. This ensures that every individual auction round completes in finite time.

D. Property 2: Conflict-free task allocation

[Theorem 2] Under the iterative assignment strategy defined in Section II-E, the system guarantees that (1) every task is assigned to a robot, and (2) no robot is assigned more than one task.

Proof: Consider the sequence of tasks $k = 1, \ldots, M$. For any task k, the Max-Consensus algorithm runs over the set of available robots $\mathcal{A}_k$. From Theorem 1, the algorithm converges to a single winner $i^* \in \mathcal{A}_k$ (assuming a deterministic tie-breaking rule resolves equal bids).

By the update rule defined in Section II-E, $\mathcal{A}_{k+1} = \mathcal{A}_k \setminus \{i^*\}$. The winner i^* is explicitly removed from the pool of eligible candidates for all future tasks $k' > k$. Consequently, it is impossible for robot i^* to bid on or be assigned a second task, guaranteeing the capacity constraint is never violated.

Furthermore, we assume the number of robots N is sufficient for the number of tasks M ($N \geq M$). The size of the availability set evolves as $|\mathcal{A}_k| = N - (k-1)$. Since $k \leq M \leq N$, it follows that $|\mathcal{A}_k| \geq 1$ for all k. Therefore, the set of available robots is never empty, and there is always at least one robot available to win each subsequent task. This guarantees that all M tasks are successfully assigned.

E. Discussion of analytical tools

The proofs above rely on the structure of the max-consensus map, which is a monotone, idempotent, max-preserving operator. The finite-time convergence property arises from graph diameter arguments, a standard result in algebraic graph theory. The uniqueness of the task allocation follows from the sequential reduction of the availability set $\mathcal{A}_k$, which transforms the complex global optimization problem into a series of strictly feasible local auctions.

IV. VALIDATION IN SIMULATION

Team contributions: Andy Tsai, Alan Cheng

To validate the theoretical properties proven in Section III, we adopted a two-stage validation strategy. First, we performed a numerical analysis using a Python-based simulation to verify the convergence and conflict-free assignment properties of the iterative consensus algorithm. Second, we implemented the system in a high-fidelity ROS 2 physics simulation to demonstrate the complete perception-decision-action pipeline.

A. A. Numerical Validation

We simulated a scenario with $N = 5$ robots and $M = 4$ tasks. To rigorously test the robustness of the Iterative Assignment Strategy (Section II-E), we initialized the system in a geometrically "unfair" configuration where Robot 0 is significantly closer to all tasks than the other agents.

Setup: The communication graph was a line topology ($0-1-2-3-4$) with a diameter $D = 4$. The max-consensus algorithm was run with a step size of $\epsilon = 0.001$ over 20 iterations for each task auction.

Results: Figure 1 illustrates the evolution of the winning bid estimates for the sequence of four tasks.

- **Convergence:** In all four auctions, the consensus variables converged to a stable value within 5 iterations. This empirically validates **Theorem 1**, as the convergence time is bounded by the graph diameter ($D = 4$).
- **Sequential Assignment:**
  - *Task 1:* Robot 0, being the closest, wins the auction.
  - *Task 2:* Although Robot 0 is still the closest agent, it is removed from the availability set $\mathcal{A}_2$. Consequently, the network converges to the next best candidate, Robot 3.
  - *Task 3 & 4:* The process repeats, assigning subsequent tasks to Robot 4 and Robot 2, respectively.

These results confirm **Theorem 2**: despite the initial geometric bias, the iterative strategy successfully enforced the capacity constraint, resulting in a conflict-free allocation where four distinct robots were assigned to the four tasks.

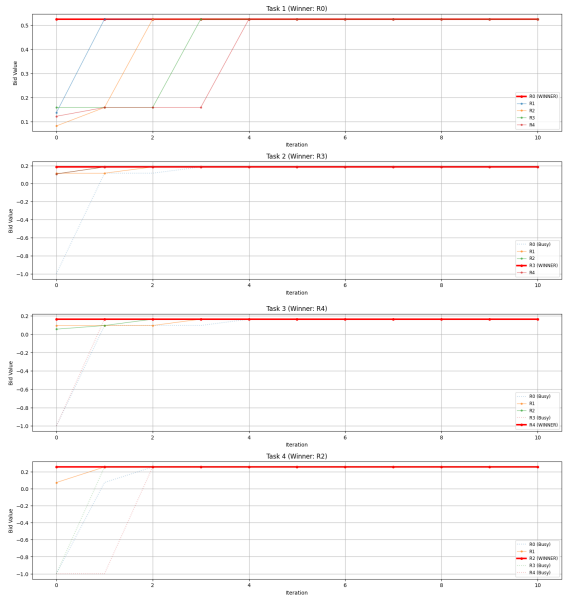

Fig. 1. Numerical validation of the Iterative Consensus Strategy. The plots show the bidding process for 4 sequential tasks. Note how the winner of a previous task (e.g., R0 in Task 1) effectively withdraws from future auctions, forcing the system to select the next best available robot.

B. B. System Validation (ROS 2, Gazebo Fortress)

We established a high-fidelity physical simulation environment in Gazebo Fortress to validate the robotic platform and navigation subsystems. The system implementation utilizes **ROS 2 Humble**.

Architecture: The proposed system architecture consists of distributed ROS 2 nodes handling three core functions:

1) **Localization:** Subscribing to ground-truth pose data (or AMCL estimates) to track robot positions in the warehouse.
2) **Navigation:** A configured `Nav2` stack responsible for global path planning and local obstacle avoidance.
3) **Consensus Interface:** A designed interface to translate the discrete task assignments (from Section IV-A) into navigation goals.

Simulation Results: The simulation environment was initialized with $N = 5$ robots and $M = 4$ tasks, matching the "unfair" geometric configuration used in the numerical analysis. The agents were deployed at starting positions $p_0 = (0.0, 2.5)$, $p_1 = (-2.0, 7.5)$, $p_2 = (-7.5, -7.5)$, $p_3 = (2.5, -5.0)$, and $p_4 = (8.0, -3.0)$. The target task locations were set to $q_0 = (1.0, 1.0)$, $q_1 = (-1.0, -1.0)$, $q_2 = (7.5, 3.0)$, and $q_3 = (-4.0, -6.0)$.

Figure 2 demonstrates the successful initialization of the fleet within the warehouse environment. The physical platform, illustrated in Fig. 4, was modeled in URDF as a differential drive robot equipped with a simulated 2D LiDAR sensor.

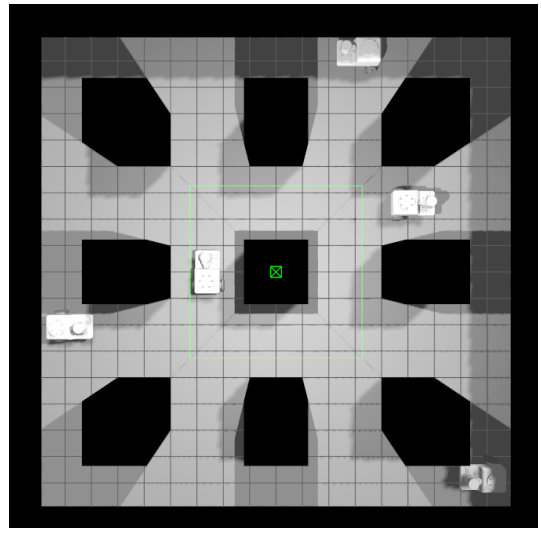

Fig. 2. Snapshot of the multi-robot simulation environment in Gazebo Fortress. The black boxes represent the storage shelves in the warehouse, and the five robots are shown initialized at the start configurations.

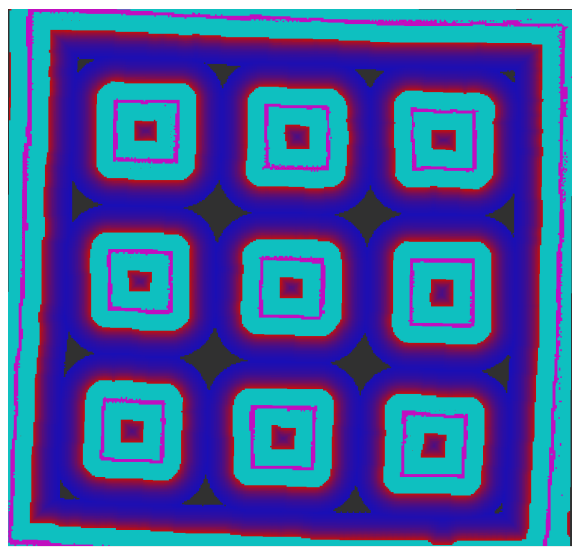

Fig. 3. Visualization of the global costmap generated by the Nav2 stack. The dark regions represent static obstacles (shelves), while the gradient (inflation layer) indicates the safety buffer zones used by the path planner to avoid collisions.

To enable autonomous motion, the warehouse environment was mapped into a static occupancy grid. Figure 3 illustrates the generated global costmap used by the Nav2 stack. The dark regions represent static obstacles (shelves), while the gradient (inflation layer) indicates the safety buffer zones used by the path planner to avoid collisions.

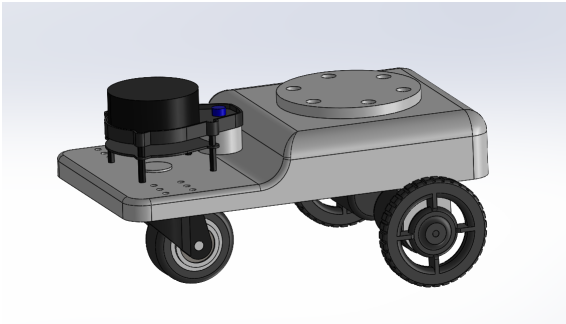

Fig. 4. The custom differential drive robot model used for validation.

Summary of Validation: The numerical analysis (Section IV-A) confirmed the correctness of the consensus logic, while the ROS 2 simulation (Section IV-B) validated the physical feasibility of the robot model, the mapping quality, and the navigation configuration. The real-time integration of these validated subsystems remains a subject for future development.

The source code for the simulation and the consensus logic is available at: Consensus warehouse robot.

STATEMENT OF AI USAGE

The authors acknowledge the use of Google Gemini to assist in this project. The AI tool was utilized to:

- Assist in the formal proof mathematical theorems.
- Assist in troubleshooting ROS 2 simulation and debug scripts for the numerical validation.
- Refine the technical writing and format the LaTeX source code for the final report.

REFERENCES

[1] M. Mesbahi and M. Egerstedt, *Graph Theoretic Methods in Multiagent Networks*. Princeton University Press, 2010.

[2] F. Bullo, J. Cortés, and S. Martínez, *Distributed Control of Robotic Networks*. Princeton University Press, 2009.

[3] A. Nedić, A. Olshevsky, and W. Shi, "Distributed Learning, Optimization, and Resource Allocation: A Unified Approach," *Foundations and Trends in Systems and Control*, vol. 7, no. 1, pp. 1–196, 2018.

[4] B. Gerkey and M. Mataric, "A Survey of the Multirobot Task Allocation Problem," *Proceedings of the IEEE*, vol. 94, no. 7, pp. 1283–1305, 2009.

[5] D. Bertsekas, "The Auction Algorithm for Assignment Problems," *Computing*, vol. 50, pp. 1–10, 1993.

[6] P. Mahato, S. Saha, and C. Sarkar, "Consensus-based Fast and Energy-Efficient Multi-Robot Task Allocation," in *Proc. IEEE Int. Conf. Robotics and Automation*, 2022.

[7] H. Chakraa, F. Guérin, E. Leclercq, and D. Lefebvre, "Optimization techniques for Multi-Robot Task Allocation problems: Review on the state-of-the-art," *Robotics and Autonomous Systems*, vol. 168, p. 104492, 2023.

[8] Canonical Ubuntu, "Making robot navigation easy with Nav2 and ROS!," YouTube Video, [Online]. Available: https://www.youtube.com/watch?v=jkoGkAd0GYk&t=456s.

[9] Gazebo Simulation, "Gz-sim Official GitHub Repository," Source Code, [Online]. Available: https://github.com/gazebosim/gz-sim/.

[10] Open Robotics, "Gazebo Documentation: Get Started," Website, [Online]. Available: https://gazebosim.org/docs/all/getstarted/.

[11] Kevin Morhaliek, "Simulation with ROS 2 Humble + Ignition Fortress (Gazebo) using the nav2 framework and Slam Toolbox," YouTube Video, [Online]. Available: https://www.youtube.com/watch?v=otlqoRIzzdI.

[12] FishROS, "Hands-on Learning ROS2 (Dongshou Xue ROS2)," Online Book, [Online]. Available: https://fishros.com/d2lros2/#/.

[13] Robotics Back-End, "ROS 2 Tutorials," YouTube Playlist, [Online]. Available: https://www.youtube.com/playlist?list=PLgG0XDQqJckkSJDPhXsFU_RIqEh08nG0V.